
← BACK TO HOME

SECURITY

IDENTITY FOR REASONING AGENTS: A RESEARCH BRIEF

JESSICA CHEN





A service account does the same thing forever, so a static, broadly scoped credential mostly works for it. An AI agent does not. It reasons, and what it needs access to changes mid-task as it interprets results and decides what to do next. A coding agent starts with read access to a repository, decides it should run the test suite, then deploy, then read a production database to debug a failing test — an



That gap, between a credential issued once and an actor whose needs evolve as it thinks, is what this event is about. Agents are a new class of actor: they hold credentials like a machine but act with the autonomy of a user, and they are multiplying inside systems never designed to govern them.

The hard problem of agent engineering is no longer capability. In production the dominant risk is not that the model says something wrong; it is that a capable agent holding standing, over-scoped credentials *does* something wrong, and the blast radius is whatever those credentials could reach. Hence the question every team answers on stage:

When your agent acts, is it acting as itself, or as you?
Where does its authority



This memo covers why that question is the hard part of agent engineering, the concepts behind it, the 1Password tooling you'll have on site, and projects worth building.

Agent Identity Build Day is an AGI House event on June 27th in Hillsborough, with speakers including Dawn Song (professor, UC Berkeley), Keith Enright (CSO at Harvey AI and former Chief Privacy Officer at Google), and HD Moore (founder of Metasploit); details and registration at [the event page](#).

MACHINE IDENTITY, AND WHY AGENTS MAKE IT HARDER

WHAT A MACHINE IDENTITY IS



service, workload, script, container, or agent — using keys, tokens, or certificates rather than a password and MFA. Machines can't answer an SMS prompt or a fingerprint scan, so the human authentication model doesn't apply; trust rests on cryptographic material and on where and how a workload runs.

These identities already outnumber human ones by roughly an order of magnitude in most enterprises (commonly cited ratios run from 45:1 to over 80:1), and their lifecycles span from certificates that live for years to workloads that exist for seconds. At that scale manual management is impossible and secrets sprawl: keys get hard-coded, copied between tools, committed to repos, and left unrotated. The average machine secret stays active



about a third of private repositories contain a plaintext secret. Agents add a new leak surface, scattering secrets into prompts, configs, and MCP server definitions.

HOW A CREDENTIAL BECOMES A BREACH

This is what most breaches actually run on. The pattern is consistent: an attacker gains a foothold, scans the environment — source, container images, logs, config — for an embedded long-lived secret, authenticates as the over-privileged machine identity that secret belongs to, and moves laterally and escalates from there.

It works because the activity looks like legitimate automation, so human-centric controls never fire. The root enabler in nearly every case is the same: a standing, over-scoped



WHY REASONING CHANGES THE PROBLEM

Agents are harder than traditional machine workloads, and the reason is reasoning. A conventional workload is "set and forget": it performs a fixed function, so a static policy fits it. A reasoning agent's access needs are not fixed; they evolve as it interprets state and chooses its next action, so a policy that was correct when the process started can be wrong a few steps later.

Zero Trust says grant only the minimum access needed, but "the minimum" is now a moving target over the life of a single process. The result is an awkward hybrid: agent access looks more like a human's evolving, context-dependent needs than a machine's fixed ones, yet it has to be granted to a



injection is a standing threat, any agent has to be treated as potentially turned against you — so the authority bound to it at any moment should be the least that is justified right then, so a hijack is contained rather than catastrophic.

WHY ATTRIBUTION IS HARD

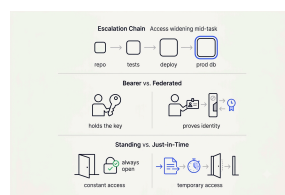
When a human errs you can trace their access and decisions; when a deterministic system fails the logs tell a linear story. Agent harm often emerges from a chain of individually-innocuous steps — a conversation that gradually walks the agent into retrieving sensitive data or invoking a tool outside its intended scope — driven by prompt manipulation, a tool misconfiguration, or an over-permissioned identity.



every action to a runtime identity: who or what acted, under which authorization, on what data, when, and from what inputs. As 1Password's security team puts it, the gap here is not governance intent but execution control. If runtime identity isn't defined and logged, attribution becomes guesswork, and containment and reporting fall apart with it.

THREE PRINCIPLES FOR AGENT ACCESS

Each maps to part of the on-stage question.



1. MINIMIZE STANDING ACCESS



- **Just-enough-privilege**
scopes a grant to a specific action or resource — write to one bucket, not s3:* — so a compromised identity can't reach beyond its job.
- **Just-in-time** grants access only on request and expires it after a short, fixed window, so there is nothing standing to steal between tasks.

Together these are what "zero standing privilege" means: by default an identity has no access, and receives narrowly-scoped, short-lived authority only when a task justifies it. Provisioning is ideally gated on attestation — the workload proves it is the thing it claims to be before anything is released.

The failure this prevents is the breach multiplier above:



leaks. The discipline runs against how teams default to working, which is to over-scope "to avoid permission errors" and never revoke. In 1Password's framing, a credential that persists is already compromised.

2. AUTHORITY IS PROVEN AT RUNTIME, NOT STORED

An identity is a set of attributes bound to a trusted issuer; a relying party trusts it by verifying a signature against the issuer's public key, the same way a browser validates a TLS certificate. There is no trusted identity without a trusted issuer.

This reframes how an agent holds authority. In the **bearer model** it holds a secret and possession is sufficient — anything that copies the secret inherits the authority, which is why long-lived keys are so dangerous.



proof, and a short-lived credential is issued for the specific task. Attestation is what makes this work without a human: signals like where a process runs and who initiated it let an issuer bind a workload to an identity automatically and in real time, and each fresh attestation can be used to re-derive access rather than rely on a standing grant. Taken to its conclusion this is "secretless" access, where the verified identity *is* the credential. Workload Identity Federation over OIDC is the deployed version, supported by GitHub Actions, AWS, GCP, and Azure.

Two disciplines work together here and are worth separating: governing the entity — the *who* — is machine identity management, and securing the credential — the *means* — is secrets management. A



verified identity plus a policy decision made at the moment of use, not a key sitting in an environment variable.

3. AN AGENT ACTS ON DELEGATED AUTHORITY AND MUST STAY ACCOUNTABLE

An agent is a delegate, not a principal. It should carry its own identity, so every action attributes to it, while acting under authority a human delegated, so accountability traces back to that person.

Because its needs evolve mid-task, a single login-time grant won't do; authority has to be re-evaluated continuously, as close as possible to each action, and constrained each time to what is justified at that moment — Zero Trust applied in real time rather than at the front door. This is



minimal current-task grant bounds what it can reach. A useful corollary is to separate trust domains that look similar but aren't — generating code and acting on production systems are different blast radii and should not share a credential scope, even for the same agent.

Accountability then depends on an audit trail where identity is inseparable from action, so a chain of steps can be reconstructed after the fact. The direction the tooling is heading is intent-based access: bind a grant to the declared purpose of a task and revoke it when the agent's behavior diverges from that purpose. Monitoring behavior against stated intent is unsettled, and a good thing to push on.

THE 1PASSWORD



1Password is sponsoring, so you'll have GA developer tools and secure AI access. From "use today" to "frontier beta":

1. **Environments and runtime injection (GA, with newer programmatic read access in beta).** Define an app's secrets in 1Password instead of a .env file and pull them at runtime via the CLI (op run --environment), the SDKs (Go, Python, JavaScript), or a Service Account for headless/CI machines with no desktop app. Locally the environment mounts as a virtual file your app reads normally — passed through a UNIX pipe, never written to disk, and untrackable



disappears. This is the default for any project that needs an API key.

2. **Desktop authentication for the SDKs (GA).** A tool or script authenticates through the desktop app's native biometric/password prompt, under a time-bound session that expires on inactivity or lock. It supports vault management, group permissions, and batch operations. Use it whenever a *person* should approve a sensitive action — the human-in-the-loop counterpart to service accounts.
3. **Codex MCP server pattern (available now).** The agent acts on secrets without seeing them. The official Environments



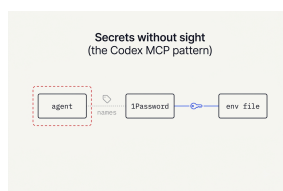
can create environments and manage variable *names* but never receives raw values; 1Password mounts them for the authorized process and requires user approval at access time. (The naive community pattern returns values through the model's context — fine for throwaway dev creds, wrong for anything real.)

4. **Credential Broker (private beta).** A workload in GitHub Actions presents a signed OIDC token, 1Password validates it against a trust policy, and only the one approved credential is released — short-lived, with full attribution (repo, branch,



credential, not the vault. Agent support is on the roadmap for later this year, which is exactly the gap your prototypes can probe.

5. **Apono (just acquired).** Where the Credential Broker governs the credential, Apono governs what the verified identity can then *do* in the target system, and for how long: just-in-time, time-bound access into AWS, Azure, GCP, Kubernetes, Snowflake, Databricks, and 200+ tools, with zero standing privilege and intent-based scoping for agents.



GitHub and Vercel in the workflow, AWS and other AI dev tools, MCP gateways like Natoma and Runlayer, AI browsers like Browserbase and Kernel. Agent access is a horizontal problem; it shows up wherever agents do real work, which is most places. Worth holding in mind when you scope a build.

PROJECT IDEAS

Starting points, not requirements. Build what you want, but be ready to answer the on-stage question about it.

AGENTS WITH A WALLET (SPENDING AUTHORITY)

- **Budget-bound procurement agent.**
Spends within a fixed budget from an approved vendor list



CONTENTS

Machine identity,
and why agents
make it harder

What a machine
identity is

How a credential
becomes a breach

Why reasoning
changes the problem

Why attribution is
hard

Three principles for
agent access

1. Minimize standing
access

2. Authority is proven
at runtime, not
stored

3. An agent acts on
delegated authority
and must stay
accountable

The 1Password stack
you'll have

Project Ideas

brokered at runtime
rather than stored, and
each transaction is
checked against budget
and allowlist before it
executes. Stripe test
mode is enough to
demo; the same
structure gates any
agent that touches
money, which is the
blocker for
procurement and
payments automation.

*Stack: an LLM agent
loop, Stripe (test
mode), the 1Password
SDK/Environments for
the payment key, a
policy/allowlist check.*

- **Per-transaction card issuance.** A booking agent that receives a payment credential scoped to one transaction and expiring after it, rather than holding a reusable card.



Access/

The Personal Chief of Staff

Orgs of Agents (Delegation Chains)

Platform and Infrastructure

Reference Materials

On-site: People and Access

At the Build Day

Stripe Issuing (test) or a mock card API, the 1Password SDK for JIT issuance, an agent loop.

AGENTS THAT RUN THE STACK (INFRASTRUCTURE ACCESS)

- **CI agent on workload identity.** An agent in GitHub Actions fetches the single credential its job needs through OIDC federation (the Credential Broker model) instead of a stored cloud key. A compromised pipeline exposes one short-lived credential, not the vault. Closest project to what the Credential Broker ships today. *Stack: GitHub Actions with OIDC, the 1Password Credential Broker, one*



- **JIT incident**

responder. On an alert, the agent requests time-bound, scoped access to a single resource, investigates, and the access revokes automatically. Models zero standing privilege for autonomous ops; the target is an on-call agent that never holds standing prod access.

Stack: an alert webhook, Apono-style JIT access, a single cloud resource (DB or K8s namespace), an agent.

- **Escalation-aware deploy agent.** Access is re-derived as the agent moves from QA to staging to production, each step gated by a fresh policy check rather than one upfront grant — a direct take on the



*pipeline, the
1Password Credential
Broker for per-stage
grants, per-stage policy
checks, an agent.*

THE PERSONAL CHIEF OF STAFF

- **Human-approved inbox/calendar operator.** Works across your tools but routes each sensitive action through the native desktop approval prompt via the SDK's desktop authentication. Access is the signed-in user's, time-bound, and revocable. The unsolved part of the assistant category is trust, not capability; this is a direct approach to it. *Stack: Gmail/Calendar APIs (or mocks), the 1Password SDK with*



- **Access lifecycle agent.** Provisions exactly the access a new project or hire requires, scoped, and revokes cleanly when done. *Stack: the 1Password SDK (vault and group management), a simple role definition, an agent.*

ORGS OF AGENTS (DELEGATION CHAINS)

- **Attenuated sub-delegation.** Agent A grants agent B a strictly narrower subset of its authority, and an action several hops deep still attributes to the original human. The least-solved problem on this list and the most relevant to multi-agent systems. *Stack: a multi-agent*



style or 1Password-issued), an audit chain that survives handoffs.

- **Brokered agent marketplace.** Agents delegate tasks to other agents, each receiving task-scoped, expiring credentials with full attribution. A small version demos cleanly; the general case is coordination infrastructure. *Stack: a multi-agent setup, the 1Password SDK for task-scoped expiring credentials, attribution logging.*

PLATFORM AND INFRASTRUCTURE

- **Agent-identity control plane.** Issue, scope, monitor, and revoke agent credentials across providers from one interface, built on



buildable on the SDK.

Stack: the 1Password SDK and Credential Broker, a web dashboard, a policy model.

- **Intent-conformance monitor.** A library that takes an agent's declared task, watches its tool calls, and halts on divergence — the runtime-safety layer implied by intent-based access. *Stack: your agent framework's tool-call hooks, an intent/policy spec, Apono's intent model as a reference design.*
- **Attribution-complete audit layer.** Structured logging that binds every tool call to a runtime identity and the human who delegated it, so a task chain can be



directly. *Stack:*
structured logging (e.g.
OpenTelemetry),
1Password runtime
identity, an agent
emitting tool-call
events.

- **Secret-sprawl scanner.** Scans repos and MCP configs for plaintext secrets and replaces them with 1Password references. Targets the sprawl problem above directly. *Stack: a repo/MCP-config crawler, the 1Password CLI/SDK, secret-reference rewriting.*

REFERENCE MATERIALS

From the 1Password team.

Developer tooling:

- [Secure your secrets at runtime with the](#)



- [Secure your desktop apps with 1Password SDKs](#)
- [Codex builds at AI speed, 1Password secures it](#)
- [How 1Password Environments makes secrets management simple for developers](#)

Enterprise offering worth a look:

- [Secrets at Runtime, Never in the Repo: 1Password Credential Broker](#), with the accompanying [blog post](#)

ON-SITE: PEOPLE AND ACCESS

From 1Password:

- **Márton Soos, Natalie Cluer, Tamara**



Identity Federation, attestation, the Credential Broker, delegation and attribution across agents, and zero standing privilege in practice.

- **Jody Heavener** — Developer Growth. Fastest path to getting the tooling wired up.

Access includes the GA developer tools and secure AI access: Environments, the CLI, the SDKs, Service Accounts, and the Codex MCP server. Schedule and registration are on the event page above.

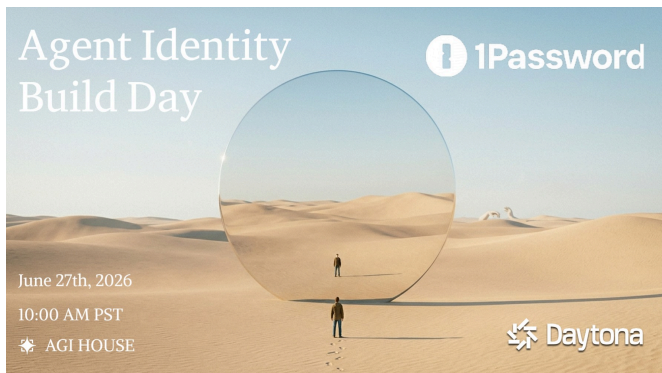
AT THE BUILD DAY

The agents that land best won't be the most capable — they'll be the ones whose authority is clear: the team



who answers when it acts.
Because agent access is a
horizontal problem, a build
that gets this right is often
the start of something larger
than the demo. See you
Saturday.

LATEST POSTS



SECURITY

Identity For Reasoning Agents: A Research Brief

BY JESSICA CHEN



DINNER SERIES

AGI Dinner Series: What's Actually Underhyped

BY ROCKY YU



© 2026 AGI HOUSE RESEARCH, ALL RIGHTS RESERVED

